컴퓨터 비전 과제 2 보고서

20201586 백기원

1. 목표

이 과제에서는 canny edge detector를 구현해보고 실제 사진의 edge를 검출해본다.

2. 구현 내용

1) gaussian_kernel

$\begin{bmatrix}-1&0&1\\-1&0&1\\-1&0&1\end{bmatrix}\begin{bmatrix}-1&-1&-1\\0&0&0\\1&1&1\end{bmatrix}$이 두 개를 만들고 이에 대해 가우시안 분포($e^{-\frac{x^2+y^2}{2\sigma^2}}$) 적용 후 정규화

```
def gaussian_kernel(size: int, sigma: float):
    x, ynp.meshgrid(np.arange(size), np.arange(size))
    center= (size1) / 2
    kernelnp.exp(-((x-center)**2 +(y-center)**2) / (2sigma**2))
    return kernel /np.sum(kernel)  # 정규화
```

2) gradient_magnitude_direction

이미지의 gradient를 구하는 sobel filter($\begin{bmatrix}-1&0&1\\-2&0&2\\-1&0&1\end{bmatrix}\begin{bmatrix}-1&-2&-1\\0&0&0\\1&2&1\end{bmatrix}$)를 구현하였다.

```
def gradient_magnitude_direction(image: np.ndarray):
    sobel_xnp.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]) #x축 sobel filter
    sobel_ynp.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]]) #y축 sobel filter
```

3) non_maximum_suppression

이전의 조건문에서 정해진 방향의 양 옆 픽셀보다 값이 크면 해당 픽셀의 값을 유지하도록 구현하였다.

```
 # 중앙 픽셀이 양 옆보다 크다면 유지
if magnitude[y, x] > max(neighbors): #neighbor보다 중앙이 큰 경우->edge
    output[y, x] = magnitude[y, x]
else:
    output[y, x] = 0
```

4) edge_tracking_by_hysteresis

weak_edge 주변에 strong_edge가 있는지 찾고 있다면 final_edges에 포함시켰다.

```
for y in range(1, height-1):
    for x in range(1, width-1):
        if weak_edges[y, x]:
            # linking, 주위에 강한 edge가 있으면 final_edges에 포함시킨다
            if np.any(strong_edges[y-1:y+2, x-1:x+2]):
                final_edges[y, x] = True
            else:
                final_edges[y, x] = False
```

3. 결과

